

Module Programmation mathématique avancée 2

Pratiques : Minima locaux via la méthode du gradient – II

Travail 1

On considère la minimisation de la fonction $f(x_1, x_2) = (x_1 - 1)^2 + 2x_2^2$, avec comme point initial $x^0 = (0, 0)^t$. Le gradient est $\nabla f(x) = (2(x_1 - 1), 4x_2)^t$; le minimum exact est donc $x^* = (1, 0)$.

```
import numpy as np
import matplotlib.pyplot as plt
import time

def f(x):
    return (x[0] - 1)**2 + 2 * x[1]**2

def gradient(x):
    return np.array([
        2 * (x[0] - 1),
        4 * x[1]
    ])

def gradient_method(x0, alpha, iterations):
    x = np.array(x0, dtype=float)
    resultats = []
    debut = time.time()

    for k in range(iterations):
        grad = gradient(x)
        d = -grad
        x_new = x + alpha * d

        resultats.append([
            k,
            x.copy(),
            x_new.copy(),
            np.linalg.norm(d),
            np.linalg.norm(x_new - x),
            np.linalg.norm(grad)
        ])

        x = x_new

    fin = time.time()
    return resultats, fin - debut

def gradient_method_tol(x0, alpha, tol, max_iter=10000):
    x = np.array(x0, dtype=float)
    k = 0
    debut = time.time()

    while np.linalg.norm(gradient(x)) > tol and k < max_iter:
```

```

    x = x - alpha * gradient(x)
    k += 1

    fin = time.time()
    return x, k, fin - debut

# Point initial
x0 = [0, 0]

# a) alpha = 1
resultats1, temps1 = gradient_method(x0, 1, 100)

# b) alpha = 0.5
resultats2, temps2 = gradient_method(x0, 0.5, 300)

print("Temps alpha = 1 :", temps1)
print("Temps alpha = 0.5 :", temps2)

```

a) Tableau pour $\alpha = 1$ (100 itérations)

Point initial : $x^0 = (0, 0)$

k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(0, 0)	(2, 0)	2.0000	2.0000	2.0000
1	(2, 0)	(0, 0)	2.0000	2.0000	2.0000
2	(0, 0)	(2, 0)	2.0000	2.0000	2.0000
3	(2, 0)	(0, 0)	2.0000	2.0000	2.0000
4	(0, 0)	(2, 0)	2.0000	2.0000	2.0000

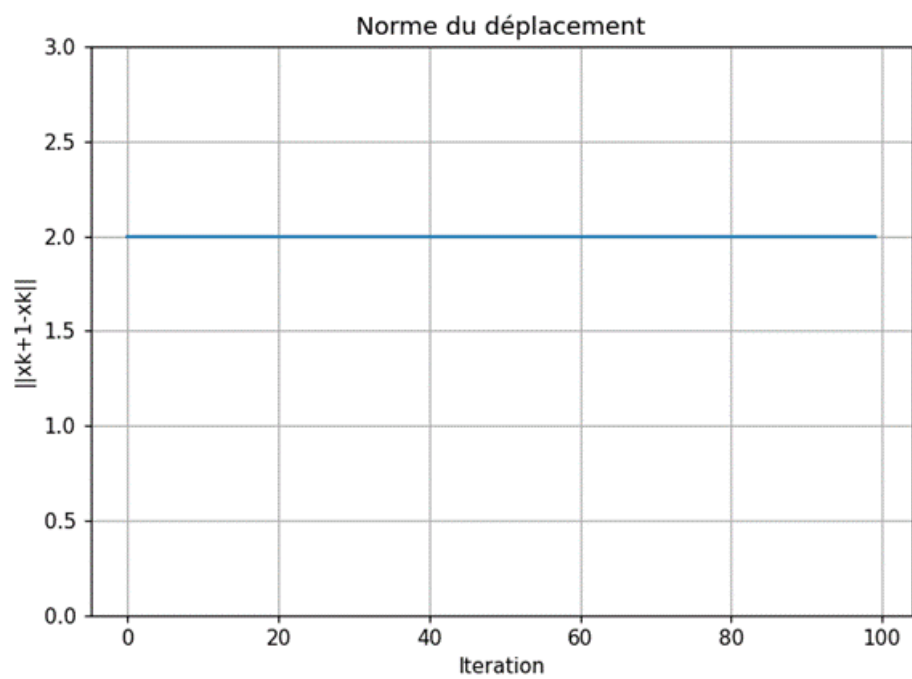
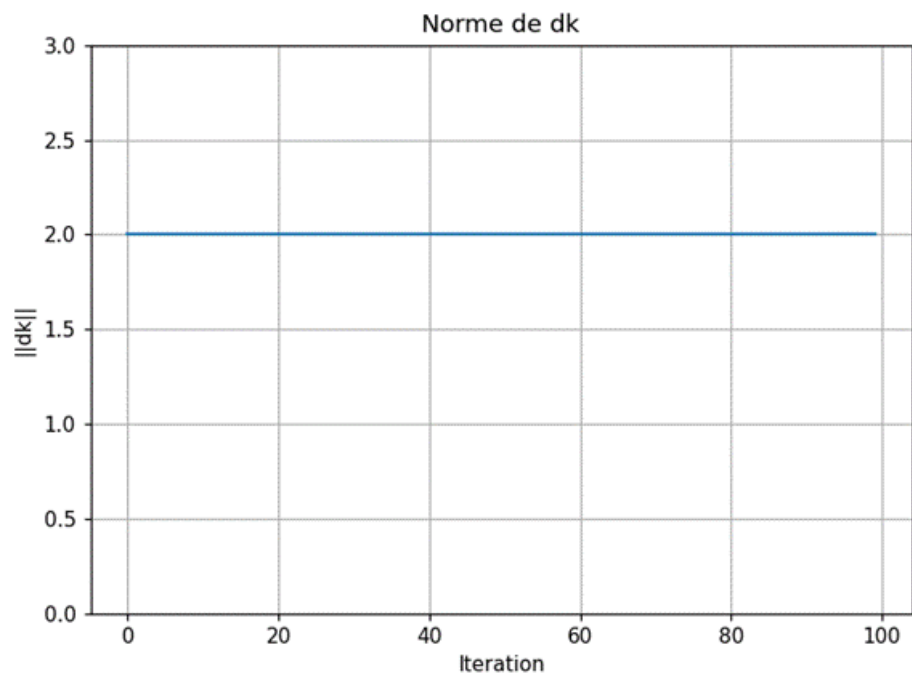
Les 100 itérations sont identiques aux 5 premières lignes. Dernier itéré : $x^{100} = (0, 0)$, avec $\|\nabla f(x^{100})\| = 2$.

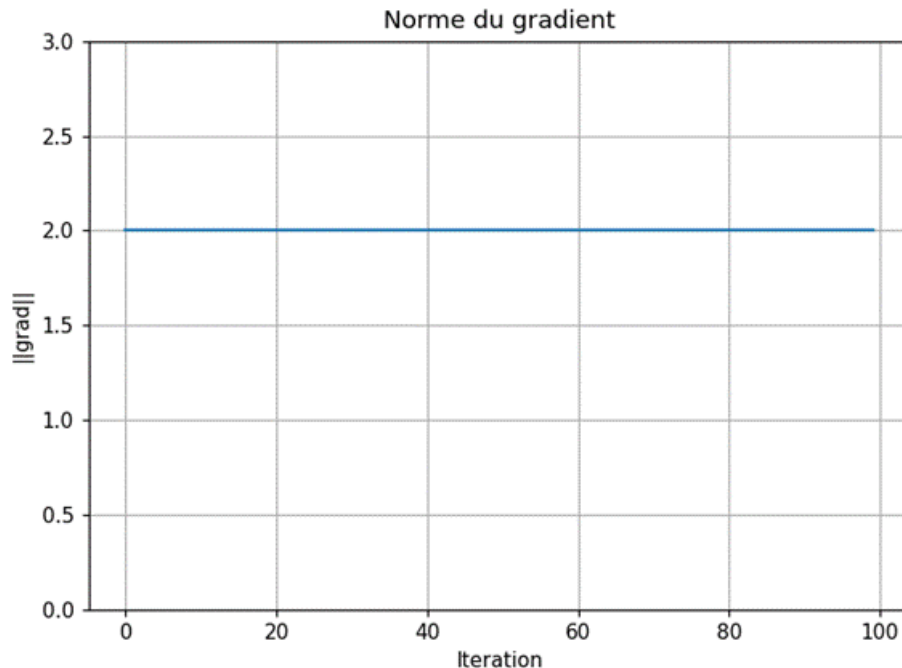
Observation

On remarque que :

- les itérés oscillent indéfiniment entre (0, 0) et (2, 0) ;
- les trois normes restent constantes, toutes égales à 2 ;
- la méthode **ne converge pas** pour $\alpha = 1$.

Représentation graphique :





Le dernier point itéré est-il un minimum local ?

Non. Le dernier itéré est $x^{100} = (0, 0)$ et $\|\nabla f(x^{100})\| = 2 \neq 0$: la condition nécessaire du premier ordre ($\nabla f(x) = 0$) n'est pas vérifiée, donc x^{100} n'est pas un point stationnaire et ne peut pas être un minimum local. En effet, avec $\alpha = 1$ la mise à jour s'écrit $x_1^{k+1} = 2 - x_1^k$: l'erreur par rapport au minimum $(1, 0)$ change de signe sans jamais diminuer, la méthode oscille au lieu de converger.

b) Tableau pour $\alpha = 1/2$ (300 itérations)

Point initial : $x^0 = (0, 0)$

k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(0, 0)	(1, 0)	2.0000	1.0000	2.0000
1	(1, 0)	(1, 0)	0.0000	0.0000	0.0000
2	(1, 0)	(1, 0)	0.0000	0.0000	0.0000
3	(1, 0)	(1, 0)	0.0000	0.0000	0.0000
4	(1, 0)	(1, 0)	0.0000	0.0000	0.0000

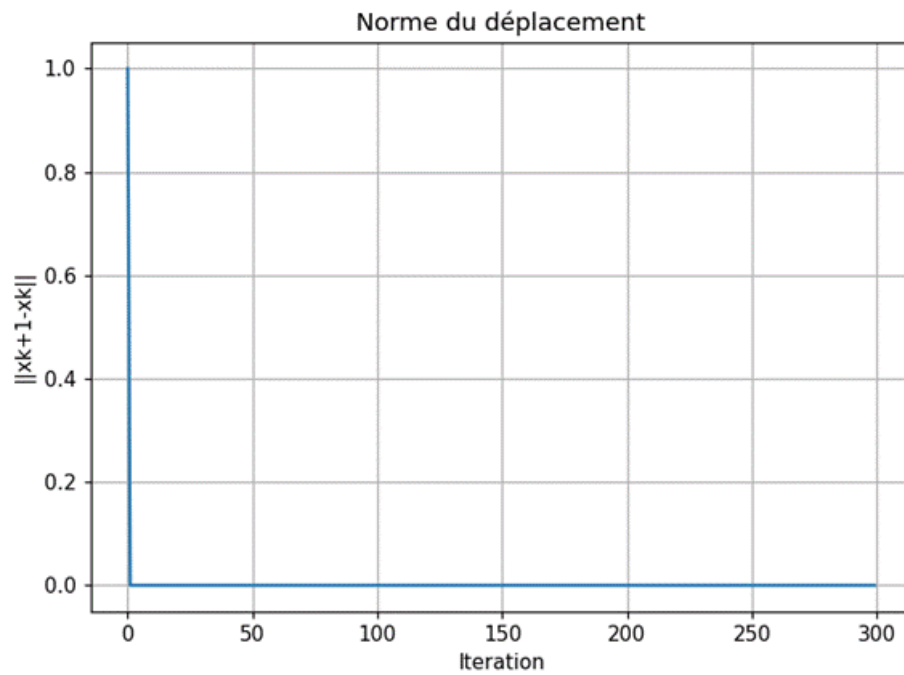
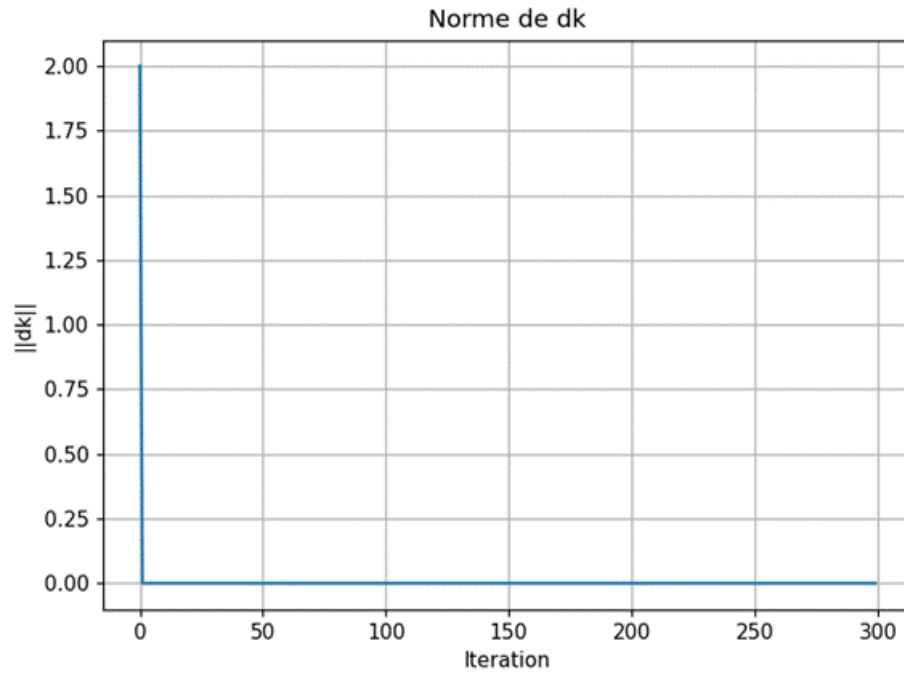
Dès la première itération, la méthode atteint $x^1 = (1, 0)$ et le gradient s'annule exactement : tous les itérés suivants restent en $(1, 0)$. Dernier itéré : $x^{300} = (1, 0)$, avec $\|\nabla f(x^{300})\| = 0$.

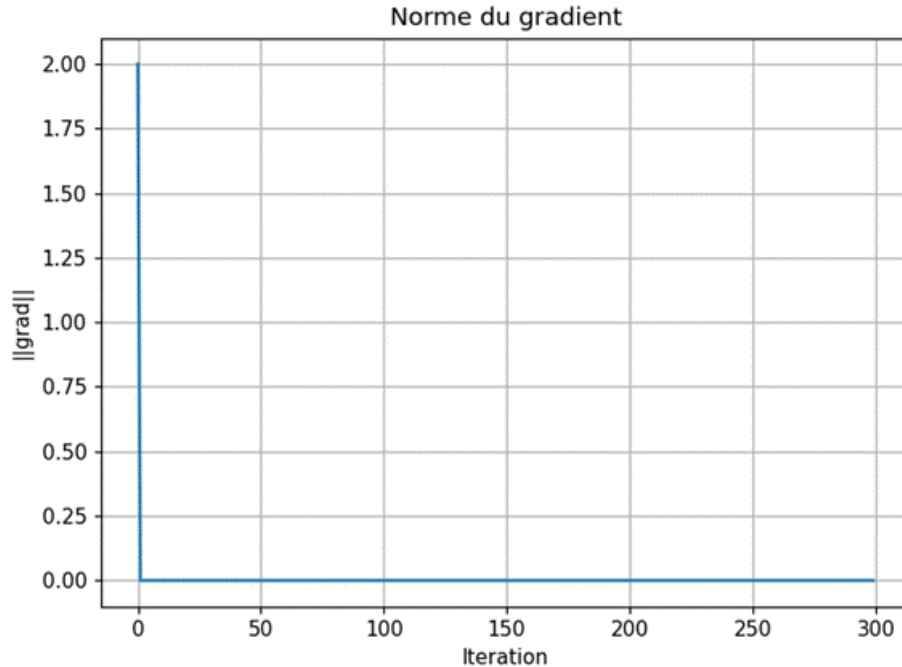
Observation

Cette fois :

- la méthode atteint le point (1, 0) en une seule itération ;
- la norme du gradient devient exactement 0 et le reste ;
- la méthode **converge**.

Représentation graphique :





Le dernier point itéré est-il un minimum local ?

Oui. Le dernier itéré $x^{300} = (1, 0)$ vérifie $\nabla f(1, 0) = (0, 0)$: la condition du premier ordre est satisfaite. De plus, la hessienne (constante) $\nabla^2 f = \text{diag}(2, 4)$ est définie positive (valeurs propres 2 et 4 > 0) : $(1, 0)$ est donc un minimum local strict. La fonction f étant convexe, c'est même le minimum global, avec $f(1, 0) = 0$.

c) Cas $\alpha = 1/2$ et $\text{tol} = 10^{-5}$

Nombre d'itérations : 1

Temps d'exécution : 9.036064147949219e-05 secondes

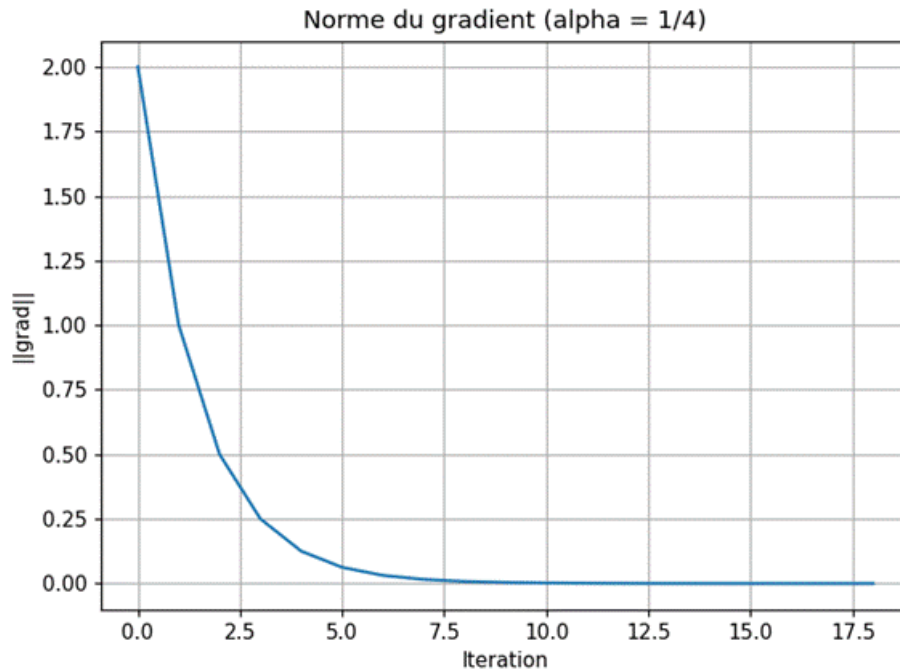
Le gradient s'annule exactement dès la première itération ($x^1 = (1, 0)$) : la précision demandée est atteinte immédiatement.

d) Cas $\alpha = 1/4$ et $\text{tol} = 10^{-5}$

Nombre d'itérations : 18

Temps d'exécution : 0.0002079010009765625 secondes

Avec $\alpha = 1/4$, la norme du gradient est exactement divisée par 2 à chaque itération : $\|\nabla f(x^k)\| = 2 \cdot (1/2)^k$. Il faut donc 18 itérations pour passer sous 10^{-5} ($\|\nabla f(x^{18})\| \approx 7.63 \times 10^{-6}$). La convergence est linéaire, de taux $1/2$.



e) Analyse comparative et recommandation

α	Comportement observé	Itérations (tol = 10^{-5})	Temps (s)
1	oscillation entre (0, 0) et (2, 0) — pas de convergence	—	—
1/2	convergence exacte vers (1, 0) en une itération	1	9.04e-05
1/4	convergence linéaire de taux 1/2	18	2.08e-04

Le comportement de la méthode dépend entièrement du pas α . Pour cette fonction quadratique, la hessienne a pour valeurs propres $\lambda_1 = 2$ et $\lambda_2 = 4$, et la théorie impose $0 < \alpha < 2/\lambda_{\max} = 1/2$ pour garantir la convergence depuis n'importe quel point initial.

- $\alpha = 1$: le facteur de contraction sur x_1 vaut $1 - \alpha\lambda_1 = -1$; l'erreur ne diminue jamais et la méthode oscille ;
- $\alpha = 1/2$: $1 - \alpha\lambda_1 = 0$, l'erreur sur x_1 est annulée en une seule itération (le facteur sur x_2 vaut -1 , ce qui poserait problème si $x_2^0 \neq 0$, mais ici $x_2^0 = 0$ est déjà optimal) ;
- $\alpha = 1/4$: les deux facteurs sont dans $]0, 1[$, la convergence est garantie mais plus lente (18 itérations).

Recommandation : pour ce problème, $\alpha = 1/2$ est le meilleur choix : convergence en 1 itération contre 18, pour un temps d'exécution environ deux fois plus court. De manière générale, le pas doit être adapté à la courbure de la fonction (condition $\alpha < 2/\lambda_{\max}$) : un pas trop grand fait osciller ou diverger la méthode, un pas trop petit la ralentit inutilement. En pratique, il est plus sûr d'utiliser un pas adaptatif (recherche linéaire) plutôt qu'un pas fixe choisi à l'aveugle.

Travail 2

On considère la minimisation de $f(x, y) = x^4 + y^4 + 2(x - y)^2$, avec comme point initial $x^0 = (1, 1)^t$. Le gradient est $\nabla f(x, y) = (4x^3 + 4(x - y), 4y^3 - 4(x - y))^t$. On redéfinit simplement f et gradient, puis on réutilise la fonction `gradient_method_tol` du Travail 1.

```
import numpy as np
import matplotlib.pyplot as plt
import time

def f(v):
    x, y = v
    return x**4 + y**4 + 2 * (x - y)**2

def gradient(v):
    x, y = v
    return np.array([
        4 * x**3 + 4 * (x - y),
        4 * y**3 - 4 * (x - y)
    ])

def gradient_method_tol(x0, alpha, tol, max_iter=10000):
    x = np.array(x0, dtype=float)
    k = 0
    debut = time.time()

    while np.linalg.norm(gradient(x)) > tol and k < max_iter:
        x = x - alpha * gradient(x)
        k += 1

    fin = time.time()
    return x, k, fin - debut

# Point initial
x0 = [1, 1]

# a) alpha = 1/2
x_a, k_a, t_a = gradient_method_tol(x0, 0.5, 1e-5)

# b) alpha = 1/4
x_b, k_b, t_b = gradient_method_tol(x0, 0.25, 1e-5)

# Affichage des résultats
print("==== a) α = 1/2 =====")
print("Solution :", x_a)
print("f(x) =", f(x_a))
print("Itérations :", k_a)
print("Temps :", t_a, "secondes")

print("\n==== b) α = 1/4 =====")
print("Solution :", x_b)
print("f(x) =", f(x_b))
print("Itérations :", k_b)
print("Temps :", t_b, "secondes")
```


a) Cas $\alpha = 1/2$ et $\text{tol} = 10^{-5}$

Premières itérations :

k	$\mathbf{x}^k$	$\mathbf{x}^{k+1}$	$\ \mathbf{d}^k\ $	$\ \mathbf{x}^{k+1} - \mathbf{x}^k\ $	$\ \nabla f(\mathbf{x}^k)\ $
0	(1, 1)	(-1, -1)	5.6569	2.8284	5.6569
1	(-1, -1)	(1, 1)	5.6569	2.8284	5.6569
2	(1, 1)	(-1, -1)	5.6569	2.8284	5.6569
3	(-1, -1)	(1, 1)	5.6569	2.8284	5.6569
4	(1, 1)	(-1, -1)	5.6569	2.8284	5.6569

Précision non atteinte : arrêt après 10000 itérations

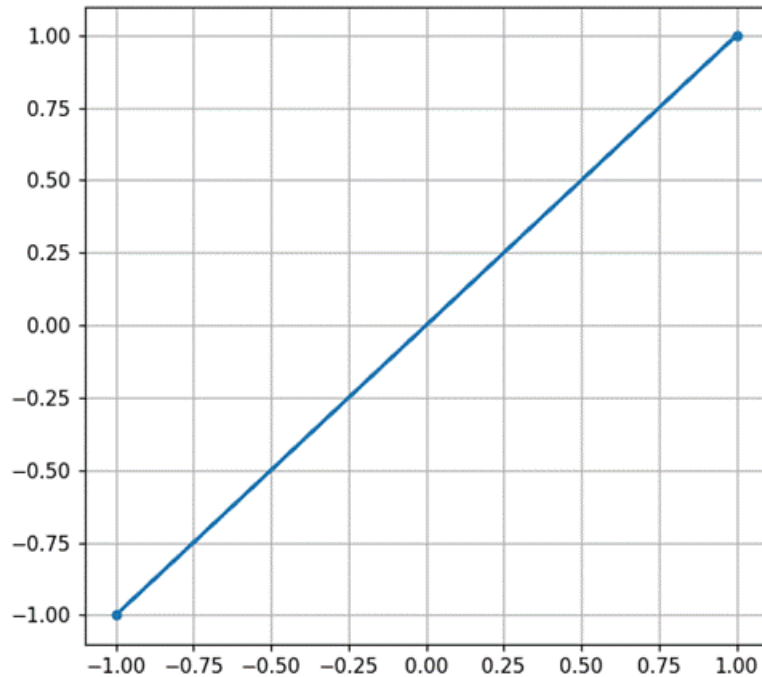
Norme du gradient finale : 5.656854249492381

Temps d'exécution : 0.09551286697387695 secondes

Observation

On remarque que :

- la méthode cycle indéfiniment entre (1, 1) et (-1, -1) ;
- en effet $\nabla f(1, 1) = (4, 4)$ donne $\mathbf{x}^1 = (1, 1) - \frac{1}{2} \cdot (4, 4) = (-1, -1)$, puis $\nabla f(-1, -1) = (-4, -4)$ ramène exactement en (1, 1) ;
- la norme du gradient reste constante ($4\sqrt{2} \approx 5.6569$) : la précision 10^{-5} n'est jamais atteinte ;
- la méthode **ne converge pas** pour $\alpha = 1/2$.



Trajectoire des itérés : le segment entre $(-1, -1)$ et $(1, 1)$ est parcouru en boucle.

b) Cas $\alpha = 1/4$ et $\text{tol} = 10^{-5}$

Nombre d'itérations : 1

Temps d'exécution : 7.772445678710938e-05 secondes

Une seule itération suffit : $x^1 = (1, 1) - \frac{1}{4} \cdot (4, 4) = (0, 0)$, et $\nabla f(0, 0) = (0, 0)$. Le point trouvé est bien un minimum : $f(x, y) = x^4 + y^4 + 2(x - y)^2 \geq 0$ pour tout (x, y) , et $f(0, 0) = 0$; $(0, 0)$ est donc le **minimum global** de f .

Conclusion : contrairement au Travail 1, c'est ici le pas $\alpha = 1/2$ qui échoue (cycle de période 2) et le pas $\alpha = 1/4$ qui réussit, en une seule itération. Le « bon » pas dépend donc de la fonction — de sa courbure au voisinage des itérés — et non d'une règle universelle.

Travail 3

On veut minimiser $f(x, y) = 5x^4 + 6y^4 - 6x^2 + 2xy + 5y^2 + 15x - 7y + 13$, avec comme point initial $x^0 = (0, 0)^t$. Le gradient est $\nabla f(x, y) = (20x^3 - 12x + 2y + 15, 24y^3 + 2x + 10y - 7)^t$, d'où $\nabla f(0, 0) = (15, -7)$ et $\|\nabla f(x^0)\| \approx 16.55$. Comme les itérés risquent d'exploser, on ajoute dans `gradient_method_tol` un test d'arrêt en cas de débordement numérique :

```
import numpy as np
import matplotlib.pyplot as plt
import time
```

```

def f(v):
    x, y = v
    return 5*x**4 + 6*y**4 - 6*x**2 + 2*x*y + 5*y**2 + 15*x - 7*y + 13

def gradient(v):
    x, y = v
    return np.array([
        20*x**3 - 12*x + 2*y + 15,
        24*y**3 + 2*x + 10*y - 7
    ])

def gradient_method_tol(x0, alpha, tol, max_iter=10000):
    x = np.array(x0, dtype=float)
    k = 0
    debut = time.time()

    while np.linalg.norm(gradient(x)) > tol and k < max_iter:
        x = x - alpha * gradient(x)
        k += 1

        # Arrêt si débordement numérique
        if not np.all(np.isfinite(gradient(x))):
            break

    fin = time.time()
    return x, k, fin - debut

# Point initial
x0 = [0, 0]

# a) alpha = 1/2
x_a, k_a, t_a = gradient_method_tol(x0, 0.5, 1e-5)

# b) alpha = 1/4
x_b, k_b, t_b = gradient_method_tol(x0, 0.25, 1e-5)

# Affichage des résultats
print("==== a)  $\alpha = 1/2$  ====")
print("Solution :", x_a)
print("f(x) =", f(x_a))
print("||grad f(x)|| =", np.linalg.norm(gradient(x_a)))
print("Itérations :", k_a)
print("Temps :", t_a, "secondes")

print("\n==== b)  $\alpha = 1/4$  ====")
print("Solution :", x_b)
print("f(x) =", f(x_b))
print("||grad f(x)|| =", np.linalg.norm(gradient(x_b)))
print("Itérations :", k_b)
print("Temps :", t_b, "secondes")

```

a) Cas $\alpha = 1/2$ et $\text{tol} = 10^{-5}$

Premières itérations :

k	$\mathbf{x}^k$	$\mathbf{x}^{k+1}$	$\ \mathbf{d}^k\ $	$\ \mathbf{x}^{k+1} - \mathbf{x}^k\ $	$\ \nabla f(\mathbf{x}^k)\ $
0	(0, 0)	(-7.5, 3.5)	16.5529	8.2765	16.5529
1	(-7.5, 3.5)	(4155.25, -517.5)	8390.4538	4195.2269	8390.4538
2	(4155.25, -517.5)	(-7.1745e+11, 1.6631e+09)	1.4349e+12	7.1745e+11	1.4349e+12
3	(-7.1745e+11, 1.6631e+09)	(3.6930e+36, -5.5197e+28)	7.3859e+36	3.6930e+36	7.3859e+36
4	(3.6930e+36, -5.5197e+28)	(-5.0364e+110, 2.0180e+87)	1.0073e+111	5.0364e+110	1.0073e+111

Divergence (débordement numérique) après 5 itérations
Temps d'exécution : 0.00015401840209960938 secondes

Observation

On remarque que :

- dès la première itération, le point saute en $(-7.5, 3.5)$, où $\|\nabla f\| \approx 8390$;
- les termes cubiques du gradient amplifient chaque pas : les itérés explosent en quelques itérations jusqu'au débordement numérique (inf) ;
- la précision 10^{-5} n'est jamais atteinte : la méthode **diverge** pour $\alpha = 1/2$.

b) Cas $\alpha = 1/4$ et $\text{tol} = 10^{-5}$

Premières itérations :

k	$\mathbf{x}^k$	$\mathbf{x}^{k+1}$	$\ \mathbf{d}^k\ $	$\ \mathbf{x}^{k+1} - \mathbf{x}^k\ $	$\ \nabla f(\mathbf{x}^k)\ $
0	(0, 0)	(-3.75, 1.75)	16.5529	4.1382	16.5529
1	(-3.75, 1.75)	(244.047, -31.1562)	999.8889	249.9722	999.8889
2	(244.047, -31.1562)	(-7.2675e+07, 1.8139e+05)	2.9070e+08	7.2675e+07	2.9070e+08
3	(-7.2675e+07, 1.8139e+05)	(1.9192e+24, -3.5808e+16)	7.6768e+24	1.9192e+24	7.6768e+24
4	(1.9192e+24, -3.5808e+16)	(-3.5346e+73, 2.7549e+50)	1.4138e+74	3.5346e+73	1.4138e+74
5	(-3.5346e+73, 2.7549e+50)	(2.2079e+221, -1.2545e+152)	inf	inf	inf

Divergence (débordement numérique) après 6 itérations
Temps d'exécution : 0.00010418891906738281 secondes

Réduire le pas de $1/2$ à $1/4$ ne change rien au comportement : le gradient initial est déjà grand ($\|\nabla f(\mathbf{x}^0)\| \approx 16.55$) et la croissance cubique du gradient rend ces deux pas beaucoup trop agressifs. La méthode diverge également, avec débordement après 6 itérations.

Analyse complémentaire : recherche d'un pas qui converge

Aucun des deux pas imposés ne permet de converger. Nous avons donc testé des pas plus petits (toujours avec $\text{tol} = 10^{-5}$) :

α	Résultat	Itérations	Temps (s)
0.05	pas de convergence : oscillations, arrêt à 500 000 itérations	—	5.03
0.02	convergence vers (-1.14205, 0.543372)	18	1.79e-04
0.01	convergence vers le même point	43	5.87e-04
0.005	convergence vers le même point	93	9.21e-04

Le meilleur pas testé est $\alpha = 0.02$: convergence en 18 itérations seulement vers $x^* \approx (-1.14205, 0.54337)$, avec $f(x^*) \approx -6.4961$. En ce point, la hessienne de f a pour valeurs propres ≈ 31.14 et 66.37 , toutes deux strictement positives : elle est définie positive et x^* est bien un minimum local. On retrouve d'ailleurs la condition de stabilité $\alpha < 2/\lambda_{\max} \approx 0.030$, qui explique l'échec de $\alpha = 0.05$ et, a fortiori, de $1/4$ et $1/2$.